

```

/* =====
PL/SQL COMPLETE EXAMPLES
Topics:
1. IF THEN
2. IF THEN ELSE
3. IF THEN ELSIF
4. FOR LOOP
5. WHILE LOOP
6. SIMPLE LOOP
7. NESTED FOR LOOP
8. PROCEDURE
9. FUNCTION
10. EXCEPTION
===== */

```

```
SET SERVEROUTPUT ON;
```

```

/* =====
1. IF THEN Example
===== */

```

```

SQL> DECLARE
    v_salary NUMBER := 800000;
BEGIN
    IF v_salary > 500000 THEN
        DBMS_OUTPUT.PUT_LINE('Salary is greater than 500,000');
    END IF;
END;
/
 2      3      4      5      6      7      8 Salary is greater than 500,000
PL/SQL procedure successfully completed.

```

```

/* =====
2. IF THEN ELSE Example
===== */

```

```

SQL> DECLARE
    v_age NUMBER := 20;
BEGIN
    IF v_age >= 18 THEN
        DBMS_OUTPUT.PUT_LINE('User is adult');
    ELSE
        DBMS_OUTPUT.PUT_LINE('User is under age');
    END IF;
END;
/
 2      3      4      5      6      7      8      9     10 User is adult
PL/SQL procedure successfully completed.

```

```

/* =====
3. IF THEN ELIF Example
===== */

```

```

SQL> DECLARE
    v_mark NUMBER := 75;
BEGIN
    IF v_mark >= 80 THEN
        DBMS_OUTPUT.PUT_LINE('Grade A');
    ELIF v_mark >= 60 THEN
        DBMS_OUTPUT.PUT_LINE('Grade B');
    ELIF v_mark >= 40 THEN
        DBMS_OUTPUT.PUT_LINE('Grade C');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Fail');
    END IF;
END;
/
 2      3      4      5      6      7      8      9     10     11     12     13     14   Grade B
PL/SQL procedure successfully completed.

```

```

/* =====
4. FOR LOOP Example
===== */

```

```

SQL> BEGIN
    FOR i IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE('FOR LOOP value: ' || i);
    END LOOP;
END;
/
 2      3      4      5      6   FOR LOOP value: 1
FOR LOOP value: 2
FOR LOOP value: 3
FOR LOOP value: 4
FOR LOOP value: 5
PL/SQL procedure successfully completed.

```

```

/* =====
5. WHILE LOOP Example
===== */

```

```

SQL> DECLARE
    v_counter NUMBER := 1;
BEGIN
    WHILE v_counter <= 5 LOOP
        DBMS_OUTPUT.PUT_LINE('WHILE LOOP value: ' || v_counter);
        v_counter := v_counter + 1;
    END LOOP;
END;
/
 2    3    4    5    6    7    8    9 WHILE LOOP value: 1
WHILE LOOP value: 2
WHILE LOOP value: 3
WHILE LOOP value: 4
WHILE LOOP value: 5

PL/SQL procedure successfully completed.

```

```

/* =====
6. SIMPLE LOOP Example
===== */

```

```

SQL> DECLARE
    v_counter NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE('SIMPLE LOOP value: ' || v_counter);

        v_counter := v_counter + 1;

        EXIT WHEN v_counter > 5;
    END LOOP;
END;
/
 2    3    4    5    6    7    8    9   10   11   12 SIMPLE LOOP value: 1
SIMPLE LOOP value: 2
SIMPLE LOOP value: 3
SIMPLE LOOP value: 4
SIMPLE LOOP value: 5

PL/SQL procedure successfully completed.

```

```

/* =====
7. NESTED FOR LOOP Example
Multiplication Table
===== */

```

```

SQL> BEGIN
    FOR i IN 1..3 LOOP
        FOR j IN 1..5 LOOP
            DBMS_OUTPUT.PUT_LINE(i || ' x ' || j || ' = ' || (i * j));
        END LOOP;

        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;
END;
/
    2      3      4      5      6      7      8      9      10    1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
-----
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
-----
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
-----

PL/SQL procedure successfully completed.

```

```

/* =====
Create Sample Table
===== */

```

```
DROP TABLE students;
```

```
CREATE TABLE students
```

```

(
    student_id NUMBER PRIMARY KEY,
    student_name VARCHAR2(100),
    mark NUMBER,
    grade VARCHAR2(10)
);

```

```

INSERT INTO students VALUES (1, 'Aung Aung', 85, NULL);
INSERT INTO students VALUES (2, 'Mg Mg', 70, NULL);
INSERT INTO students VALUES (3, 'Su Su', 45, NULL);
INSERT INTO students VALUES (4, 'Hla Hla', 30, NULL);

```

```
COMMIT;
```

```

/* =====
8. PROCEDURE Example
Procedure to update student grade
===== */
CREATE OR REPLACE PROCEDURE update_student_grade
(
    p_student_id IN NUMBER
)
AS
    v_mark students.mark%TYPE;
    v_grade VARCHAR2(10);
BEGIN
    SELECT mark
    INTO v_mark
    FROM students
    WHERE student_id = p_student_id;

    IF v_mark >= 80 THEN
        v_grade := 'A';
    ELSIF v_mark >= 60 THEN
        v_grade := 'B';
    ELSIF v_mark >= 40 THEN
        v_grade := 'C';
    ELSE
        v_grade := 'Fail';
    END IF;

    UPDATE students
    SET grade = v_grade
    WHERE student_id = p_student_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Student grade updated successfully');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

```
SQL> SELECT * FROM students;
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK GRADE
```

```
1
```

```
Aung Aung
```

```
85
```

```
2
```

```
Mg Mg
```

```
70
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK GRADE
```

```
3
```

```
Su Su
```

```
45
```

```
4
```

```
Hla Hla
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK GRADE
```

```
30
```

```
/* Run Procedure */
```

```
SQL> BEGIN
```

```
    update_student_grade(1);
```

```
    update_student_grade(2);
```

```
    update_student_grade(3);
```

```
    update_student_grade(4);
```

```
END;
```

```
/
```

```
2      3      4      5      6      7 Student grade updated successfully
```

```
Student grade updated successfully
```

```
Student grade updated successfully
```

```
Student grade updated successfully
```

```
PL/SQL procedure successfully completed.
```

```
/* Check Result */
```

```
SQL> SELECT * FROM students;
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK GRADE
```

```
1  
Aung Aung  
85 A
```

```
2  
Mg Mg  
70 B
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK GRADE
```

```
3  
Su Su  
45 C
```

```
4  
Hla Hla
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK GRADE
```

```
30 Fail
```

```
/* =====  
9. FUNCTION Example  
Function to return grade by mark  
===== */  
CREATE OR REPLACE FUNCTION get_grade  
(  
    p_mark IN NUMBER  
)  
RETURN VARCHAR2  
AS  
    v_grade VARCHAR2(10);  
BEGIN  
    IF p_mark >= 80 THEN
```

```

        v_grade := 'A';
    ELSIF p_mark >= 60 THEN
        v_grade := 'B';
    ELSIF p_mark >= 40 THEN
        v_grade := 'C';
    ELSE
        v_grade := 'Fail';
    END IF;

    RETURN v_grade;
END;
/

```

/* Run Function */

```

SQL> SQL> BEGIN
      DBMS_OUTPUT.PUT_LINE('Grade is: ' || get_grade(90));
      DBMS_OUTPUT.PUT_LINE('Grade is: ' || get_grade(55));
END;
  2      3      4      5 /
Grade is: A
Grade is: C

PL/SQL procedure successfully completed.

```


/* Use Function in SELECT */

```
SQL> SELECT
      student_id,
      student_name,
      mark,
      get_grade(mark) AS calculated_grade
FROM students;
```

```
2      3      4      5      6
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK
```

```
CALCULATED_GRADE
```

```
1
```

```
Aung Aung
```

```
85
```

```
A
```

```
STUDENT_ID
```

```
STUDENT_NAME
```

```
MARK
```

```
CALCULATED_GRADE
```

```
2
```

```
Mg Mg
```

```
70
```

```
B
```

```

/* =====
10. EXCEPTION Example
===== */

```

```

SQL> DECLARE
    v_student_name students.student_name%TYPE;
BEGIN
    SELECT student_name
    INTO v_student_name
    FROM students
    WHERE student_id = 100;

    DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_student_name);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No student found with this ID');

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Query returned more than one row');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected Error: ' || SQLERRM);
END;
/
  2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
 18  19  20  21 No student found with this ID

PL/SQL procedure successfully completed.

```

```

/* =====
11. Complete Example: Loop + Procedure + Function
Update all students grade using FOR LOOP
===== */

```

```

SQL> BEGIN
    FOR rec IN
    (
        SELECT student_id
        FROM students
    )
    LOOP
        update_student_grade(rec.student_id);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('All student grades updated');
END;
/
  2   3   4   5   6   7   8   9  10  11  12  13
Student grade updated successfully
Student grade updated successfully
Student grade updated successfully
Student grade updated successfully
All student grades updated

PL/SQL procedure successfully completed.

```

```
/* Final Result */
```

```
SQL> SELECT * FROM students;
```

```
STUDENT_ID
```

```
-----
```

```
STUDENT_NAME
```

```
-----
```

```
MARK GRADE
```

```
-----
```

```
1
```

```
Aung Aung
```

```
85 A
```

```
2
```

```
Mg Mg
```

```
70 B
```

```
STUDENT_ID
```

```
-----
```

```
STUDENT_NAME
```

```
-----
```

```
MARK GRADE
```

```
-----
```

```
3
```

```
Su Su
```

```
45 C
```

```
4
```

```
Hla Hla
```

```
STUDENT_ID
```

```
-----
```

```
STUDENT_NAME
```

```
-----
```

```
MARK GRADE
```

```
-----
```

```
30 Fail
```